

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

Six public Java test sets

Where this fits

Our own 24 hard files showed that a live model can recover cases the pattern checker misses. Those files were written by us, so they are not enough.

This note reports the same simple checker on six public Java sets that other people published. The live-model numbers on these sets are not in the table, and the reason is below.

The idea we started with

A result on files we wrote can show that the system runs. It cannot show that the system works on code we did not write. Public suites are the check against that bias.

We also started from a rule we had already broken once: the question must not contain the answer. Any model score gathered before that rule was enforced is not usable, even if the number looks interesting.

Why we chose this way

We chose six named sets rather than one, because one suite has one style. Juliet is teaching code. OWASP Benchmark is a scored suite. Securibench is small servlet examples. Find Security Bugs is detector test code. Vul4J and the CVEfixes slice are closer to real fixes. Together they show a range. Together they are still not "all Java".

We scored the pattern checker on the full ingested sets, and we did not relabel nearby CWE ids to match our six. A missing weakness is reported as not present. Relabeling would invent recall.

The earlier live-model samples on these sets are withdrawn. The prompts still contained tokens such as `method_bad`, `FLAW` comments, or `real=true`. Those tokens tell the model the label. Quoting those F1 numbers would repeat a contaminated experiment.

What we actually did

Juliet is a large teaching suite from NIST. OWASP Benchmark is a scored Java suite. Securibench Micro is a small set of servlet examples. Find Security Bugs contributes detector test code. Vul4J and the CVEfixes slice are closer to real project bugs, but we only keep the Java files that match the weaknesses we study.

We ran the pattern checker on every file we ingested. F1 is again the combined score of false alarms and missed bugs. The best F1 here is 0.435, on 79 Find Security Bugs files. The lowest is 0.072, on 119 Securibench files. Juliet is the largest set, 20,728 files, and its F1 is 0.316.

A larger set is not an easier set. The checker is a list of text patterns. It is not CodeQL, and it is not a commercial scanner that follows data through the program.

We did run the live model earlier on small samples of these sets. Those scores are withdrawn. The questions still contained giveaway words, such as a method named `bad` or a comment

that said the code was flawed. A fair test must not show the answer inside the question.

The public sets also do not all contain every weakness we study. When a set has no files for a weakness, we say "not present". We do not rename a nearby weakness to fill the gap.

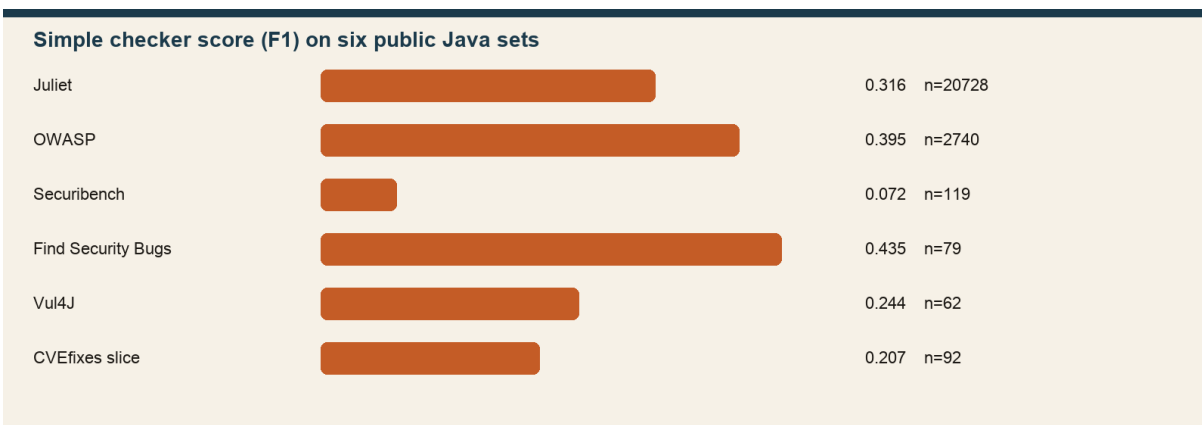
A concrete example

Juliet has thousands of small teaching files. A file name often says whether it is the bad version or the good version. If we leave the word bad in the question, a model can cheat. That is why the old model scores are withdrawn.

Securibench is small, 119 files, and the simple checker almost never matches the labels. F1 0.072 means the pattern list does not fit those servlets. A bigger Juliet score of 0.316 is still only a modest pattern match, not a success.

Find Security Bugs is the highest bar in the picture, 0.435, and it is only 79 files. We do not average the six rows. Each set is its own result.

Picture



Each bar is the simple checker's F1. Longer is better. None of these bars is a model score.

The numbers

Test set	Files	Simple checker F1
Juliet Java	20728	0.316
OWASP Benchmark	2740	0.395
Securibench Micro	119	0.072
Find Security Bugs	79	0.435
Vul4J	62	0.244
CVEfixes Java slice	92	0.207

Read each row on its own. Do not average them into one project score. Do not mix them with the 0.923 score on our 24 files.

What we concluded, and why

The pattern checker is weak on public code. That is a real result, and it is the fair baseline.

The next measurement is a live-model score on Juliet after the giveaway words are hidden. We will use matched unsafe and safe files, not a single mixed file. The conclusion of this note is that the pattern checker is weak on public code, and the old model samples cannot be saved.

What we are not saying

Do not quote the withdrawn model scores. The simple checker is not CodeQL and not a commercial product.